

Reviewer Pack — Minimal Order of Geometric Flows and DPLL Proof Tree Depth

Entry 987e88c6ae25 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 01:45:33 UTC

Minimal Order of Geometric Flows and DPLL Proof Tree Depth

Entry ID: 987e88c6ae25 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 01:45:33 UTC

1. Conjecture

Field A (mathematical branch): Geometric Flow Theory **Field B** (complexity object): Complexity Theory (DPLL Search Trees)

Statement:

For every instance of a Boolean satisfiability problem with n variables, the minimal order of a geometric flow on a suitably defined metric space corresponding to the problem is linearly correlated with its DPLL proof tree depth. Equivalently, if $o(G)$ represents the minimal order of the geometric flow for an instance G and $d(G)$ represents its DPLL proof tree depth, then $o(G) = \Theta(d(G))$.

Rationale (proposer's reasoning):

Geometric flow theory, particularly in the study of mean curvature flows, has shown potential to capture complex geometric structures. By mapping Boolean satisfiability problems into metric spaces and studying the flow behavior, we may expose hidden complexity-theoretic properties that are not evident through standard analysis methods. This could reveal new insights into proof complexity.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0e8339881f9bb211

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if, for all $n \leq 40$, the mean of the ratio between the computed minimal order of the geometric flow and the DPLL proof tree depth across 30 random seeds is within a threshold of $\pm 10\%$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -geometric flow AND satisfiability problem AND DPLL proof tree-Boolean satisfiability AND geometric flows AND order complexity-metric space AND geometric flow theory AND DPLL proof tree depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        return [[random.choice([1, -1]) * (i + 1) for i in range(n)] for _ in range(n)]

    def dpll(cnf):
        if not cnf:
            return True
        clause = next((c for c in cnf if any(x > 0 for x in c)), [])
        if not clause:
            return False
        p = abs(clause[0])
        if any(dpll([c for c in cnf if all(x != p or x == -y for y in c)]) for x in [p, -p]):
            return True
        return False

    def geometric_flow_order(cnf):
        n = len(cnf)
        distance_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                distance_matrix[i][j] = distance_matrix[j][i] = sum(abs(x - y) for x, y in zip(cnf[i], cnf[j]))

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
```

```

        if abs(A[j][i]) > abs(A[max_row][i]):
            max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
    return A

reduced_matrix = gaussian_elimination(distance_matrix)
rank = sum(1 for row in reduced_matrix if any(x != 0 for x in row))
return rank

n = random.randint(5, 40)
cnf = generate_cnf(n)
dpll_depth = len(dpll(cnf)) - 1
flow_order = geometric_flow_order(cnf)

return {
    "metric_name": "Ratio of Flow Order to DPLL Depth",
    "metric_value": Fraction(flow_order, dpll_depth) if dpll_depth > 0 else None,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": flow_order == dpll_depth,
    "counterexample": "" if flow_order == dpll_depth else f"Flow Order: {flow_order}, DPLL Dep
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] and r["counterexample"] != "" for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369ef7a4.py", line 79, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369ef7a4.py", line 62, in run_trial
    dpll_depth = len(dpll(cnf)) - 1
                  ^^^^^^^^^^^^^

```

TypeError: object of type 'bool' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required computations and verify the conjecture. | next: Re-run the test with proper error handling to ensure it completes without crashing. If the test passes, re-evaluate the conjecture based on the results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14779
2	propose	ollama_remote	glm4:latest	0	0	16747
3	preregistration	ollama_remote	glm4:latest	0	0	13928
4	novelty	ollama_remote	glm4:latest	0	0	8385
5	novelty	ollama_remote	glm4:latest	0	0	8950
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20471
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7614
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12801
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10477
10	judge	ollama_remote	glm4:latest	0	0	11718

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125870 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/987e88c6ae25.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/987e88c6ae25.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/987e88c6ae25.tar.gz` (if generated)